except in extreme circumstances. However, this reduces the run time's ability to manage its resources, potentially degrading the overall user experience.

Applications that update regularly but only rarely or intermittently need user interaction are good candidates for implementation as Services. MP3 players and sports-score monitors are examples of applications that should continue to run and update without a visible Activity.

Further examples can be found within the software stack itself: Android implements several Services, including the Location Manager, Media Controller, and Notification Manager.

5.2 Creating and Controlling Services

In the following sections you'll learn how to create a new Service, and how to start and stop it using Intents and the startService method. Later you'll learn how to bind a Service to an Activity to provide a richer communications interface.

5.2.1 Creating a Service

To define a Service, create a new class that extends Service. You'll need to override onBind and onCreate:

```
import android.app.Service;
    import android.content.Intent;
    import android.os.IBinder;
    public class MyService extends Service {
      @Override
      public void onCreate() {
        // TODO: Actions to perform when service is created.
      }
      @Override
      public IBinder onBind(Intent intent) {
         // TODO: Replace with service binding implementation.
         return null;
      }
  }
```

In most cases, you'll also want to override `onStartCommand`. This is called whenever the Service is started with a call to `startService`, so it may be executed several times within a Service's lifetime. You should ensure that your Service accounts for this.

The `onStartCommand` handler replaces the `onStart` event that was used prior to Android 2.0. By contrast, it enables you to tell the system how to